Traffic Sign Recognition Using Convolutional Neural Networks: A Comparison of Custom CNN and Transfer Learning Approaches

Dhyey Jariwala

CS 5100 - Foundations of Artificial Intelligence

Northeastern University

April 2026

1 Introduction

Accurate traffic sign recognition is a foundational capability in the development of intelligent transportation systems and autonomous vehicles. Road signs serve as the primary communication channel between road infrastructure and drivers, conveying critical information such as speed limits, warnings about road hazards, and regulatory instructions. In the context of autonomous driving, where no human driver is present to interpret the environment, robust and accurate sign classification is essential for safe decision-making. A misclassification - for instance, failing to recognize a stop sign or misreading a speed limit - can have serious consequences for vehicle safety and road users alike.

The challenge of traffic sign recognition is not trivial. Signs must be recognized under a wide variety of real-world conditions, including changes in lighting throughout the day, partial occlusion by other objects, motion blur caused by vehicle speed, weathering and physical damage to signs, and varying camera angles and distances. These factors introduce significant intra-class variation, meaning that different images of the same sign can look quite different, while certain signs from different classes may appear visually similar. For example, speed limit signs for 30 km/h, 50 km/h, and 70 km/h share nearly identical structure and differ only in the numeral displayed, making fine-grained classification particularly demanding.

Convolutional Neural Networks (CNNs) have emerged as the dominant approach for image classification tasks due to their ability to automatically learn hierarchical spatial features directly from raw pixel data, without the need for hand-crafted feature engineering. Lower layers of a CNN learn low-level features such as edges and color gradients, while deeper layers learn increasingly abstract and task-specific representations. This hierarchical feature learning makes CNNs particularly well-suited for tasks like traffic sign recognition, where the relevant visual patterns span multiple levels of abstraction. Early work by Sermanet and LeCun (2011) demonstrated that multi-scale CNN architectures could achieve human-level performance on traffic sign classification [6], establishing deep learning as the state-of-the-art

approach for this problem.

The German Traffic Sign Recognition Benchmark (GTSRB), introduced by Stallkamp et al. (2011), has since become the standard evaluation dataset for traffic sign recognition systems [2]. The benchmark contains over 50,000 images spanning 43 distinct traffic sign categories, all captured under real-world driving conditions with natural variation in lighting, perspective, resolution, and occlusion. Its scale, diversity, and publicly available ground truth labels have made it the reference dataset for comparing different recognition approaches.

This project develops and systematically evaluates two deep learning approaches to traffic sign classification on GTSRB: first, a custom CNN architecture designed from scratch specifically for this task, and second, a transfer learning approach using a ResNet18 model pretrained on the ImageNet dataset [3]. Transfer learning is a widely used technique in which a model trained on a large, general-purpose dataset is adapted for a more specific downstream task, with the intuition that low-level visual features learned from millions of natural images may transfer usefully to new domains. The goal of this comparison is to determine whether a compact, domain-specific architecture can match or outperform a much larger pretrained model when adapted to the traffic sign domain, and to understand the conditions under which each approach is advantageous.

Prior work has directly addressed this comparison in the context of GTSRB. Mithra et al. (2023) compared a domain-specific CNN against a fine-tuned VGG16 model, finding that their custom CNN achieved 99.16% validation accuracy compared to only 87.09% for VGG16 [1]. This surprising result - that a smaller, purpose-built architecture outperformed a much deeper pretrained model - motivated the present study. However, the present work differs from Mithra et al. in three important ways. First, we use ResNet18 rather than VGG16 as the transfer learning baseline; ResNet18's residual connections address the vanishing gradient problem and enable more effective training of deep architectures, potentially making it a stronger baseline than VGG16. Second, we adopt a structured two-phase fine-tuning strategy

in which the backbone is initially frozen and only the classification head is trained, before all layers are unfrozen for full fine-tuning; this is a more principled approach than end-to-end training from random initialization. Third, our evaluation framework is more comprehensive, incorporating per-class precision, recall, and F1-score alongside overall accuracy, allowing us to identify which specific sign categories pose the greatest challenge to each model.

2 Methods

2.1 Dataset

All experiments in this project used the GTSRB dataset [2], accessed via the PyTorch `torchvision` library [5]. The dataset consists of 39,209 training images and 12,630 test images distributed across 43 traffic sign classes. The images were originally captured from real driving footage and vary considerably in size, resolution, lighting conditions, viewing angle, and background context. The dataset is moderately class-imbalanced: some categories, such as speed limit signs, contain over 700 test images, while rarer sign types contain as few as 60 test images. This imbalance is relevant because it affects the reliability of per-class performance estimates and the ease of learning rare categories.

Figure 1 shows representative examples drawn from 16 of the 43 sign classes, illustrating the diversity of shapes, colors, and symbols present in the dataset.

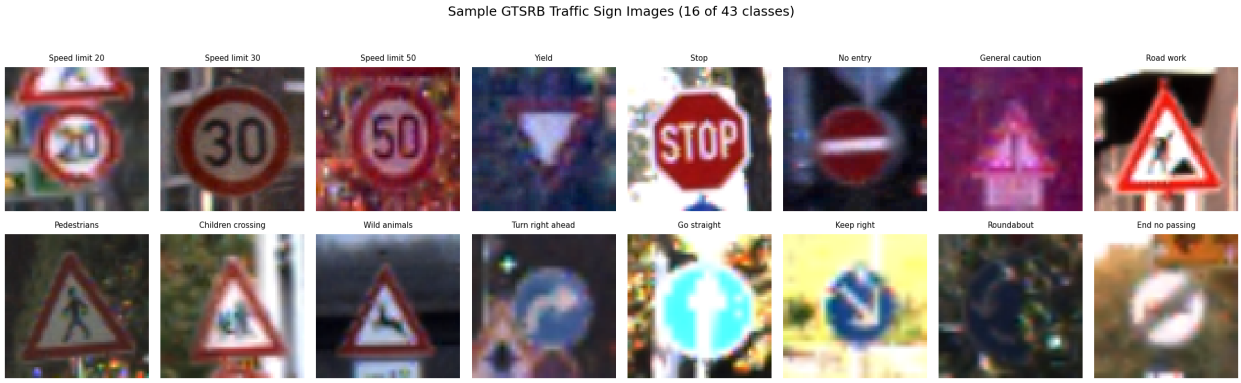

Figure 1: Representative samples from the GTSRB dataset across 16 of 43 traffic sign classes. Each image has been resized to 64×64 for display. The dataset spans prohibitory, warning, and mandatory sign types, exhibiting significant variation in shape, color, and symbol design.

2.2 Preprocessing and Augmentation

All images were resized to a fixed resolution of 32×32 pixels prior to training and evaluation. Pixel values were normalized channel-wise using the mean (0.3337, 0.3064, 0.3171) and standard deviation (0.2672, 0.2564, 0.2629), which were computed from the GTSRB training set. Normalization ensures that each color channel has zero mean and unit variance, which stabilizes gradient-based optimization and accelerates convergence.

To improve the generalization of the trained models and reduce overfitting, training-time data augmentation was applied to the training subset. Specifically, random rotations of up to $\pm 15^\circ$ were applied to simulate variation in camera angle. Random affine translations of up to 10% of image width and height were used to account for variation in sign placement within the frame. Color jitter with brightness and contrast variation of ± 0.3 was applied to simulate changes in illumination. No augmentation was applied at evaluation time, ensuring that validation and test metrics reflect performance on unmodified images.

The full GTSRB training set was split into a training subset of 22,644 samples and a validation subset of 3,996 samples, using an 85/15 random split. The original 12,630-sample test set provided by the benchmark was held out entirely during training and used only for final

performance evaluation.

2.3 Model 1: Custom CNN Architecture

The first model is a custom CNN designed from scratch for the traffic sign classification task. The architecture consists of three sequential convolutional blocks. Each block contains a 3×3 convolutional layer with same-padding, followed by batch normalization, a Rectified Linear Unit (ReLU) activation function, and 2×2 max-pooling with stride 2. The number of convolutional filters increases progressively across the three blocks - 32, 64, and 128 channels respectively - enabling the network to learn increasingly complex feature representations. The three successive pooling operations reduce the spatial resolution from the input size of 32×32 to 4×4 by the end of the convolutional stage.

The convolutional feature maps are then flattened into a vector of length $128 \times 4 \times 4 = 2{,}048$ and passed through two fully connected layers. The first fully connected layer projects this representation to a hidden dimension of 256 units, followed by a ReLU activation and dropout regularization with a dropout rate of 0.5, which randomly zeros half of the hidden units during training to prevent co-adaptation of features. The second fully connected layer maps the 256-dimensional hidden representation to the 43-class output logits. The total number of trainable parameters in the custom CNN is **629,291**.

The model was trained for 30 epochs using the cross-entropy loss function and the Adam optimizer [4] with an initial learning rate of 10^{-3}. A step learning rate scheduler was applied, decaying the learning rate by a multiplicative factor of 0.5 every 10 epochs. Mini-batches of size 64 were used throughout training.

2.4 Model 2: ResNet18 Transfer Learning

The second model is based on ResNet18 [3], an 18-layer deep residual network originally trained on the ImageNet large-scale visual recognition dataset. ResNet18 introduces residual

connections, also called skip connections, which allow the gradient signal to bypass one or more layers and flow directly to earlier layers during backpropagation. This architectural innovation addresses the vanishing gradient problem that limits the trainability of very deep networks, enabling effective optimization of architectures that would otherwise be difficult to train. The pretrained ResNet18 weights encode rich visual representations learned from over one million natural images across 1,000 object categories.

For adaptation to the GTSRB task, the final fully connected layer of ResNet18 - originally a 512-to-1000 projection for ImageNet classification - was replaced with a new 512-to-43 linear layer initialized with random weights. All other layers retained their pretrained ImageNet weights at the start of training.

Training proceeded in two distinct phases. In Phase 1, spanning 10 epochs, all backbone parameters were frozen and only the 43-class output head (comprising 22,059 trainable parameters) was trained. This phase used the Adam optimizer with a learning rate of 10^{-3}. The purpose of Phase 1 is to allow the randomly initialized classification head to adapt to the GTSRB label space before any modification of the pretrained backbone weights, preventing the large initial gradients from the random head from corrupting the learned representations in the backbone. In Phase 2, spanning 20 epochs, all layers were unfrozen and the entire network was fine-tuned end-to-end. A reduced learning rate of 10^{-4} was used for Phase 2 to ensure that the pretrained backbone weights were updated gently rather than overwritten. A step scheduler decayed this learning rate by a factor of 0.5 every 7 epochs.

2.5 Evaluation Methodology

Both models were evaluated on the held-out GTSRB test set of 12,630 images. The primary evaluation metric was overall classification accuracy, defined as the fraction of test images correctly classified. To provide a more nuanced picture of model behavior, macro-averaged and weighted precision, recall, and F1-score were also computed. Macro-averaging computes

each metric independently per class and then averages across all 43 classes with equal weight, giving a sense of performance on rare classes. Weighted averaging computes a weighted average proportional to each class's support, reflecting overall practical performance. Per-class F1-scores were examined to identify specific sign categories that posed the greatest challenge to each model. Confusion matrices were generated for both models to visualize systematic misclassification patterns. Training and validation accuracy and loss curves were analyzed to characterize the convergence behavior of each model and to assess the presence of overfitting. All experiments were implemented in Python using PyTorch and torchvision [5], and were executed on an NVIDIA T4 GPU via Google Colab.

3 Results

3.1 Custom CNN

The custom CNN achieved a **test accuracy of 95.87%** on the 12,630-sample GTSRB test set, with a macro-averaged F1-score of 0.934 and a weighted F1-score of 0.958. The best validation accuracy during training was 99.55%, reached at epoch 26 of 30.

Figure 2 shows the training and validation accuracy and loss curves over the 30 training epochs. The model converged rapidly and stably. Validation accuracy climbed sharply from 58.5% at epoch 1 to above 97% by epoch 10, driven by the model's early learning of broad discriminative features. Accuracy continued to improve more gradually through epoch 26 before plateauing, as the model refined finer distinctions between visually similar classes. Critically, training and validation losses declined consistently in parallel throughout all 30 epochs, and no divergence between training and validation performance was observed. This pattern indicates that the regularization strategies employed - batch normalization and 0.5 dropout in the classifier - were effective in preventing the model from memorizing training examples at the expense of generalization.

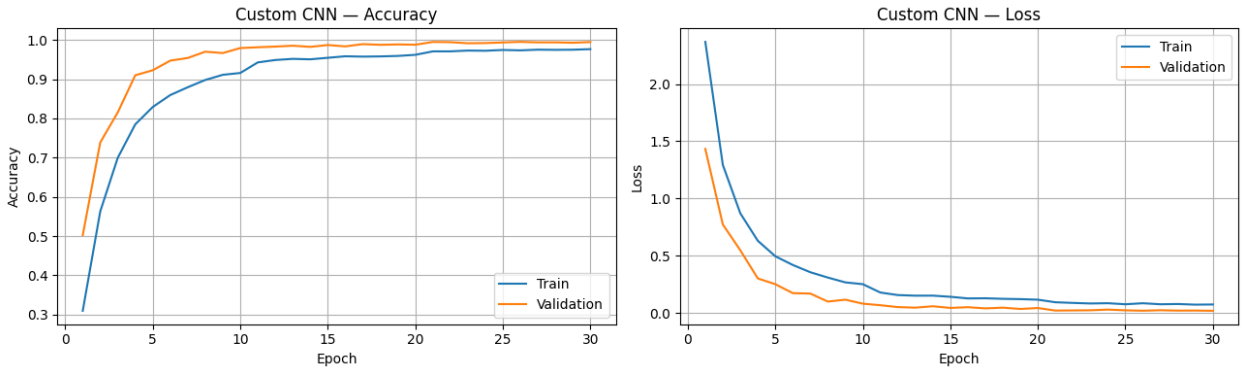

Figure 2: Custom CNN training and validation accuracy (left) and cross-entropy loss (right) over 30 epochs. The learning rate was decayed by a factor of 0.5 at epochs 10 and 20, visible as changes in the rate of loss decrease.

The model performed strongly across the majority of the 43 sign classes. Several classes achieved near-perfect classification: class 37 (go straight or left, F1: 1.000), class 10 (no passing for vehicles over 3.5 metric tons, F1: 0.999), and class 16 (vehicles over 3.5 metric tons prohibited, F1: 0.993). These high-performing categories are visually distinctive and have consistent representations across the dataset, making them relatively straightforward to classify. In contrast, the weakest-performing classes were class 30 (end of all speed and passing limits, F1: 0.678), class 41 (end of no passing, F1: 0.681), and class 27 (pedestrians, F1: 0.786). These classes are visually similar to neighboring categories - for instance, the "end of" restriction signs closely resemble their corresponding active restriction signs - and many of them appear with low frequency in the test set, with as few as 60 samples in some cases. Low sample counts reduce the statistical reliability of per-class metrics and may reflect genuine difficulty in recognizing underrepresented categories. The confusion matrix in Figure 3 confirms that misclassifications are concentrated among visually similar classes, particularly within the speed limit and restriction sign families.

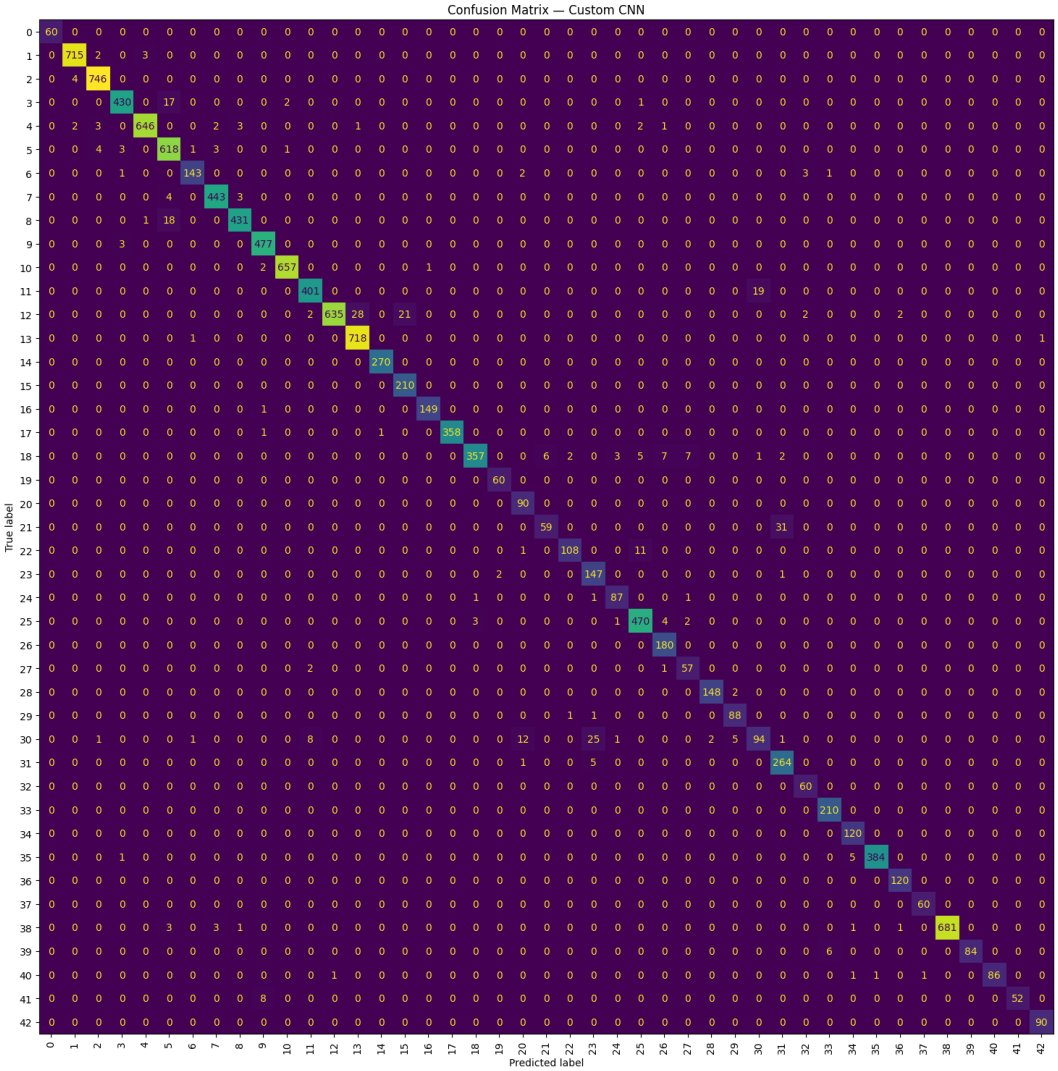

Figure 3: Confusion matrix for the custom CNN on the GTSRB test set (12,630 samples, 43 classes). Each row represents the true class and each column the predicted class. The strong diagonal indicates generally accurate classification, with off-diagonal errors concentrated among visually similar sign categories.

3.2 ResNet18 Transfer Learning

The fine-tuned ResNet18 achieved a **test accuracy of 86.83%** on the GTSRB test set, with a macro-averaged F1-score of 0.814 and a weighted F1-score of 0.868. The best validation accuracy was 99.55%, reached at epoch 19 of Phase 2.

Figure 4 shows the convergence curves across both training phases. Phase 1 (frozen back-

bone) showed notably limited progress. Despite 10 epochs of training, validation accuracy reached only approximately 42%, and the training loss remained relatively high. This result indicates that the frozen ImageNet feature representations, without any adaptation to traffic sign structure, are insufficient to drive accurate classification in this domain. The pretrained ResNet18 backbone was designed to recognize natural image textures and object parts, which differ substantially from the bold geometric shapes and symbolic content of traffic signs. The classification head, trained in isolation on top of frozen features, could not compensate for this domain mismatch.

Upon unfreezing all layers at the start of Phase 2, the model's performance improved dramatically. Within the first two fine-tuning epochs, validation accuracy jumped from approximately 42% to approximately 90%, a gain of nearly 50 percentage points. This rapid improvement confirms that the pretrained ResNet18 features become highly useful once they are allowed to adapt to the traffic sign domain, and that the gradient information from the GTSRB training data is sufficient to effectively guide this adaptation. Accuracy continued to improve steadily through Phase 2, reaching a best validation accuracy of 99.55% at epoch 19 before stabilizing.

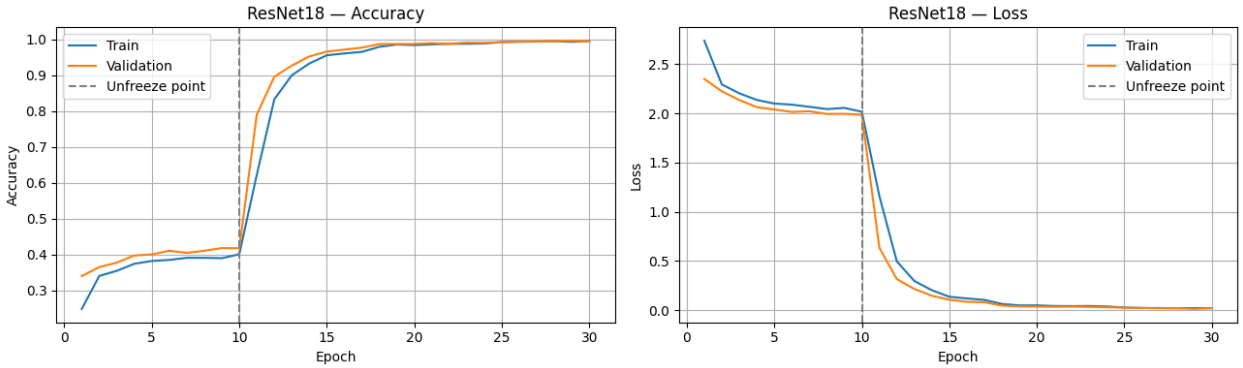

Figure 4: ResNet18 training and validation accuracy (left) and cross-entropy loss (right) over 30 total epochs (10 Phase 1 + 20 Phase 2). The dashed vertical line marks the epoch at which all backbone layers were unfrozen for full fine-tuning. The dramatic improvement immediately following unfreezing is clearly visible.

A notable observation is the large gap between ResNet18's validation accuracy (99.55%) and its test accuracy (86.83%), a discrepancy of nearly 13 percentage points. In comparison, the custom CNN's best validation accuracy (99.55%) and test accuracy (95.87%) differ by only approximately 3.7 percentage points. This substantially larger gap for ResNet18 suggests that the fine-tuned model overfit more severely to the validation distribution, learning patterns specific to the validation split that did not generalize equally well to the held-out test set. The weakest-performing classes for ResNet18 include class 21 (double curve, F1: 0.467), class 0 (20 km/h speed limit, F1: 0.469), class 41 (end of no passing, F1: 0.494), and class 19 (dangerous curve left, F1: 0.497). Several of these overlap with the custom CNN's weak classes, suggesting that these categories are intrinsically difficult regardless of model architecture. The confusion matrix for ResNet18 is shown in Figure 5.

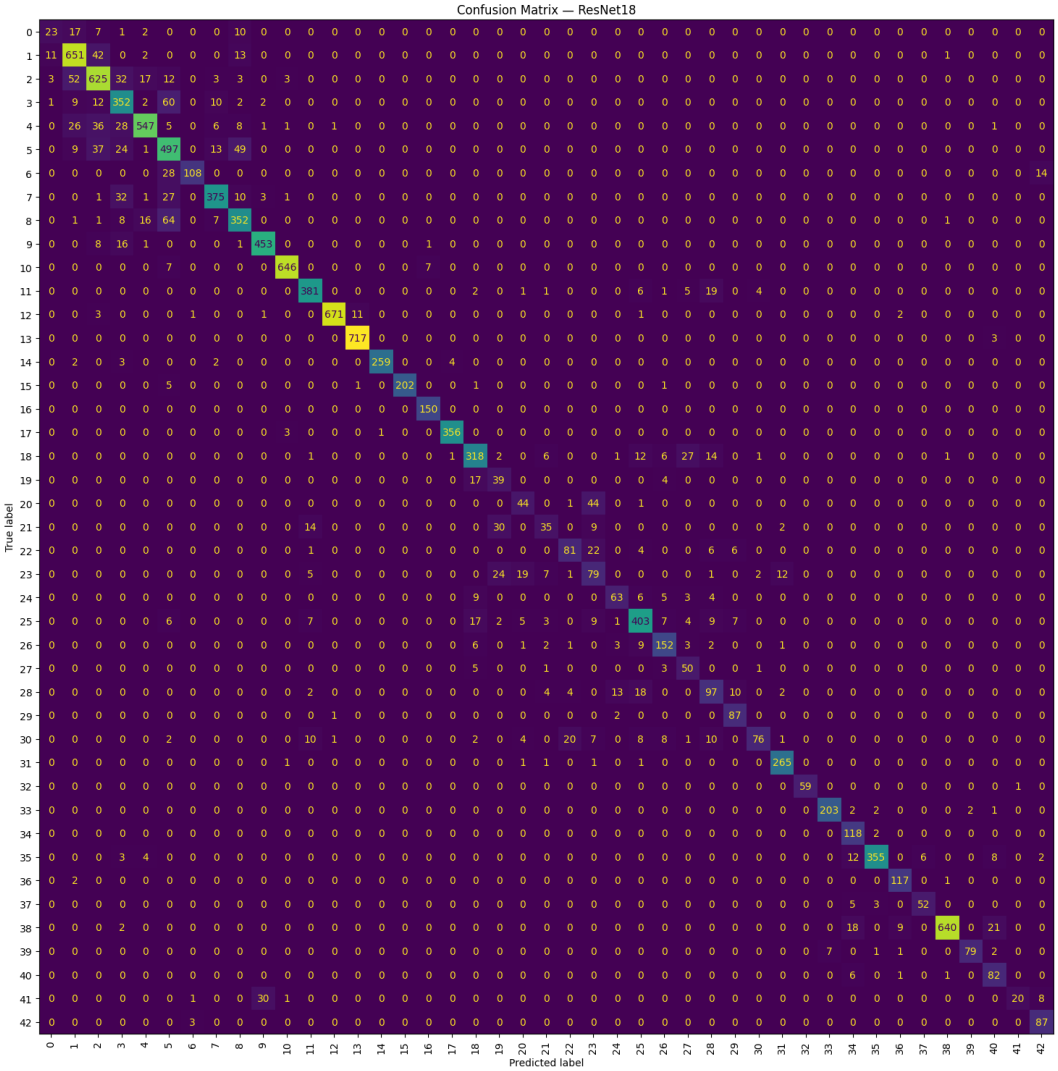

Figure 5: Confusion matrix for ResNet18 on the GTSRB test set. Compared to the custom CNN, ResNet18 exhibits more off-diagonal errors, particularly among rare and visually similar sign categories, consistent with its lower overall test accuracy.

3.3 Comparative Summary

Table 1 presents a side-by-side comparison of the two models across all key evaluation metrics. The custom CNN outperforms ResNet18 on every test set metric despite having approximately 18 times fewer trainable parameters. Both models achieve nearly identical best validation accuracy (99.55%), but the custom CNN generalizes substantially better to the unseen test set.

Table 1: Performance comparison of Custom CNN and ResNet18 on the GTSRB test set (12,630 images, 43 classes).

Metric	**Custom CNN**	**ResNet18**
Test Accuracy	95.87%	86.83%
Macro-Averaged F1	0.934	0.814
Weighted F1	0.958	0.868
Best Validation Accuracy	99.55%	99.55%
Val–Test Accuracy Gap	3.68%	12.72%
Total Trainable Parameters	629,291	11,689,512
Epochs to ∼97% Val Acc	∼10	∼16 (Phase 2)

4 Conclusions

This project demonstrates that a carefully designed, compact custom CNN substantially outperforms a fine-tuned ResNet18 on the GTSRB traffic sign classification task, achieving 95.87% test accuracy compared to 86.83% for ResNet18. This result is consistent with the findings of Mithra et al. (2023), who similarly observed that a domain-specific CNN outperformed VGG16 transfer learning on the same benchmark [1], and suggests that the advantage of domain-specific architectures over pretrained models may be a general phenomenon in the traffic sign recognition domain rather than an artifact of any particular experimental setup.

Several factors likely contribute to the custom CNN's superior performance on the test set. The most fundamental explanation concerns the domain gap between ImageNet and GTSRB. ResNet18 was pretrained on natural photographs - images of animals, objects, and scenes characterized by complex textures, natural lighting, and irregular shapes. Traffic signs, by contrast, are characterized by bold geometric outlines (circles, triangles, octagons), uniform color regions (red, blue, yellow), and standardized symbolic content that bears little

visual resemblance to natural images. The features learned from ImageNet may therefore be suboptimal as a starting point for traffic sign classification, even after fine-tuning. At the small input resolution of 32×32 pixels used in this study, ResNet18's deeper convolutional layers have limited spatial detail to refine, further reducing the benefit of its richer pretrained representations.

A second important factor is the difference in generalization behavior between the two models. The custom CNN exhibits a validation-to-test accuracy gap of only 3.68 percentage points, while ResNet18's gap is 12.72 percentage points. This disparity suggests that ResNet18 overfit more significantly during fine-tuning, learning patterns in the validation set that did not transfer to the test set. The custom CNN's relatively small parameter count (629,291 versus ResNet18's 11.7 million) may provide a form of implicit regularization that limits overfitting, as the model does not have sufficient capacity to memorize training examples and is forced to learn more generalizable representations.

Both models shared a common set of weakest-performing classes, including class 21 (double curve), class 27 (pedestrians), class 30 (end of all speed and passing limits), and class 41 (end of no passing). The shared nature of these failures suggests that these categories present inherent challenges independent of model architecture. Visually, these signs closely resemble other categories in color and shape, differing primarily in fine details of the interior symbol. At 32×32 resolution, these fine details may be lost or ambiguous, making accurate classification difficult for both models. Additionally, these classes are among the most infrequent in the dataset, providing less training signal and resulting in less reliable per-class metrics.

Several directions for future work could address the limitations identified in this study. Increasing the input resolution to 48×48 or 64×64 pixels would preserve more fine-grained detail and may particularly benefit ResNet18, whose deeper feature hierarchy is better suited to higher-resolution inputs. Applying focal loss, which down-weights easy examples and up-weights hard, rare examples during training, could improve per-class performance on un-

derrepresented sign categories. A more principled fine-tuning strategy for ResNet18, such as gradual unfreezing - thawing backbone layers one group at a time from the output end toward the input, progressively exposing more pretrained weights to adaptation - may reduce the overfitting observed in Phase 2 and close the gap between validation and test accuracy. Finally, from a practical deployment perspective, the custom CNN's compact size of 629K parameters represents a significant engineering advantage over ResNet18's 11.7 million parameters, as it requires less memory, less computational power, and can be deployed on resource-constrained embedded systems without model compression techniques. Future work could also explore model compression methods such as quantization and pruning for both architectures, with the goal of developing systems suitable for real-time deployment in autonomous vehicle hardware.

Works Cited

1. K. M. Mithra, P. A. George, S. J. Abit Sai, S. Abhishek, and T. Anjali, "Transfer Learning and Custom CNNs to Advance Traffic Sign Detection," *Proceedings of the 3rd International Conference on Smart Generation Computing, Communication and Networking (SMART GENCON)*, Karnataka, India, December 2023, IEEE. https://www.researchgate.net/publication/378575563_Transfer_Learning_and_Custom_CNNs_to_Advance_Traffic_Sign_Detection

2. J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, "The German Traffic Sign Recognition Benchmark: A Multi-Class Classification Competition," *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, 2011, pp. 1453–1460. https://www.ini.rub.de/upload/file/1470692848_f03494010c16c36bab9e/StallkampEtAl_GTSRB_IJCNN2011.pdf

3. K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition,"

Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.
https://arxiv.org/abs/1512.03385

4. D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015.
https://arxiv.org/abs/1412.6980

5. A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
https://arxiv.org/abs/1912.01703

6. P. Sermanet and Y. LeCun, "Traffic Sign Recognition with Multi-Scale Convolutional Networks," *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, 2011.
https://sermanet.github.io/papers/sermanet-ijcnn-11.pdf